

A Three-Filter Machine (Layer 3) — Yes/No events

Layer 3 is a **filter on a filter on a filter**. **Yes = 1, No = 0**. **F1** scans the **raw** string → results make **Outcome-1**. **F2** scans **Outcome-1** → results make **Outcome-2**. **F3** scans **Outcome-2** → final go-ahead. A real trade needs all three.

Worked example — F1 = 01 F2 = 101 F3 = 01

Stage 1 — F1 = 01 scans the RAW string. Each spot is highlighted; the event AFTER a spot is its result → builds Outcome-1:

raw = 0 1 1 0 1 0 1 1 0 1 0 1 1 0 1 1 0 1 0 1 1 0 1

Outcome-1 = 1 0 1 0 1 1 0 1

Stage 2 — F2 = 101 scans OUTCOME-1. Where it matches (blue), those spots are ARMED; their next result → builds Outcome-2:

Outcome-1 = 1 0 1 0 1 1 0 1

Outcome-2 = 0 1

Stage 3 — F3 = 01 scans OUTCOME-2 → matches → fully armed → next armed spot FIRES (event 23).

Where does Outcome-2 come from in the RAW string?

The two **green** digits of Outcome-2 are **not** next to each other in the raw string — they are "twice-filtered." Each one is the result of an **F2-armed spot**. Tracing them back:

raw = 0 1 1 0 1 0 1 1 0 1 0 1 1 0 1 1 0 1 0 1 0 1 1 0 1

pos = 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3

orange = an armed spot (event 10 and event 15). **green** = its result on the very next event (event 11 and event 16) — these two results ARE the two digits of Outcome-2 = **01**. So Outcome-2's "0" came from event 11, and its "1" came from event 16.

In one line: raw **01** → Outcome-1 **101** → Outcome-2 **01** → FIRE. Each filter is stricter, so real trades are rarer and more selective than Layer 1 or Layer 2.

Map it to your world. Renko: Yes = up brick, No = down brick. Fixed slice: Yes = target first, No = stop first. The three strings are computed purely from the Yes/No pattern; whether a real trade opens also depends on real-world conditions (live feed, margin, broker fills, regulations, connection). Layer 3 needs a longer warm-up — here 23 events to reach the first trade.